

Week 9 Profiling & Stress Audit Report

System Stress Audits, Multi-Threading Checks & Resource Optimizations

Project: Week 1 Project / Phase 2

Runtime: Windows (win32), Node.js v24.11.1, 12 CPU cores

Audit process: Node process PID 25984 serving port 3000

1. Objectives

The Week 9 audit evaluated baseline process resources, memory behavior under a 1,000-event stress burst, garbage-collection behavior, active thread usage, telemetry endpoint responsiveness, worker processing, and the responsive telemetry dashboard.

2. Baseline Resource Audit

Metric	Baseline
Node process	PID 25984
CPU time	2.71875 s
Working Set	65,011,712 bytes (~65.01 MB)
Paged Memory	74,301,440 bytes (~74.30 MB)
Virtual Memory	1,278,861,312 bytes (~1.279 GB)
Port 3000	Listening

3. Memory & Garbage-Collection Audit

After submitting 1,000 events to POST /api/stress-test, the same Node process showed a Working Set of 66,269,184 bytes (~66.27 MB), Paged Memory of 76,632,064 bytes (~76.63 MB), and Virtual Memory of 1,287,774,208 bytes (~1.288 GB). The Working Set increased by approximately 1.26 MB from baseline, indicating a relatively small physical-memory change during this test.

A standalone Node garbage-collection check using --expose-gc reported heapUsed of 3.74 MB before global.gc() and 3.97 MB afterward. RSS changed from about 41.31 MB to 42.29 MB. The test demonstrates that an explicit GC call does not necessarily reduce the process RSS immediately; the runtime may retain allocated memory for reuse.

4. Multi-Thread Diagnostics

The audited Node process reported 14 active OS threads. The host environment reported 12 CPU cores. The process also reported 239 handles. The thread count is not equivalent to CPU-core usage; Node.js and the operating system can maintain multiple runtime/system threads.

5. Responsiveness & Stress Tests

Test	Observed result
50 GET /metrics requests	50/50 completed; 1.319 s total (~26.4 ms/request)
1,000-event POST /api/stress-test	HTTP 200; 113.75 ms request time
Worker events processed	2,100 cumulative stress_burst events
Final worker queue	0

Peak simulated stress latency	1,000 ms
Average simulated stress latency	495.19 ms
Worker processing time	1.9939464 s total / 2,100 events (~0.95 ms/event)

6. Prometheus Telemetry Findings

The metrics endpoint exposed Node.js runtime metrics, HTTP request counters/histograms, worker counters, queue size, stress latency histograms, and processing-duration histograms. At the final verification point, resident memory was 69,529,600 bytes (~69.53 MB), maximum recorded event-loop lag was approximately 73.8 ms, and three major GC events had accumulated for approximately 23.9 ms total. The stress-burst latency histogram recorded 2,100 events with a maximum bucket of 1,000 ms.

7. Dashboard Verification

The responsive telemetry dashboard loaded successfully at /telemetry.html. It displayed Worker Events = 2,100, Peak Stress Latency = 1,000 ms, Memory Usage = 66.36 MB, Queue Size = 0, populated time-series charts, and the status message 'Telemetry connected'.

8. Findings & Recommendations

- **Memory:** The 1,000-event test produced only a small Working Set increase (~1.26 MB) relative to baseline.
- **Worker queue:** The queue returned to 0 after processing, indicating no events remained stuck at the verification point.
- **Processing:** Worker processing averaged approximately 0.95 ms/event based on the Prometheus cumulative sum/count.
- **Event loop:** Maximum recorded lag was approximately 73.8 ms; this should be monitored during heavier workloads.
- **Process hygiene:** Earlier audits showed multiple Node processes on the machine. For repeatable profiling, run only the intended application process.
- **Telemetry:** Keep the metrics endpoint and dashboard enabled during future stress runs so performance changes can be compared consistently.

9. Verified Results Summary

Overall result: The Week 9 audit successfully demonstrated a functioning telemetry pipeline, asynchronous worker processing, responsive HTTP telemetry, stable queue draining, and measurable runtime/resource statistics under a 1,000-event stress workload. The results provide a baseline for future optimization comparisons.